Lab-1

Introduction

Hadoop is an open-source framework that enables distributed storage and processing of large datasets using a cluster of commodity hardware. It follows a Master-Slave architecture consisting of a NameNode that manages metadata and DataNodes that store actual data. Hadoop is widely used for big data processing due to its scalability, fault tolerance, and high throughput capabilities. The framework primarily consists of HDFS (Hadoop Distributed File System) for storage and MapReduce for parallel data processing.

This report details the setup, installation, and verification of a simple Hadoop cluster on Jetstream2. The objective was to deploy an instance, install Hadoop, configure it properly, and ensure it operates as expected. Screenshots have been included to demonstrate key steps.

Instance Creation

We began by creating an instance on Jetstream2 using the virtual image "E516 Ubuntu22 UFW Enabled." The instance type selected was m3.small, providing a balanced configuration for our setup.

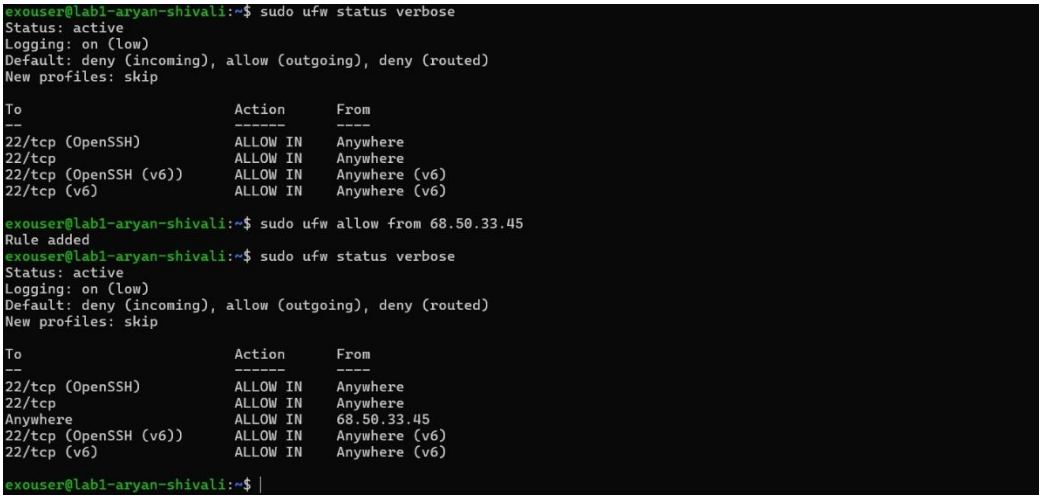
Configuring UFW

To ensure proper security and allow access from our laptop, we executed the following commands:

$ sudo ufw allow from <Laptop_IP>
$ sudo ufw enable
$ sudo ufw status verbose

The verbose flag in the ufw status verbose command provides detailed information about the active firewall rules, including default policies, allowed connections, and logging status. This helps preserve security by allowing us to audit and confirm only trusted connections are permitted while ensuring unauthorized access is blocked.

A screenshot of the configured UFW rule has been attached.



Hadoop Installation and Configuration: -

Hadoop was installed following the steps from https://medium.com/@abhikdey06/apache-hadoop-3-3-6-installation-on-ubuntu-22-04-14516bceec85. The installation was performed using the command:

wget https://dlcdn.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz

To set up Hadoop, we created the necessary directories for the NameNode and DataNode and started the Hadoop cluster using:

mkdir -p ~/hadoopdata/hdfs/{namenode,datanode}
start-all.sh

To verify that the cluster started successfully, we checked the status of key Hadoop processes using the jps command: jps. The screenshot showing NodeManager, DataNode, Jps, NameNode, ResourceManager, and SecondaryNameNode has been attached.

```
2025-02-08 22:19:57,708 INFO namenode.FSImage: Allocated new BlockPoolId: BP-1524823924-10.3.34.219-1739053197694
2025-02-08 22:19:57,735 INFO common.Storage: Storage directory /home/hadoop/hadoopdata/hdfs/namenode has been successfully formatted.
2025-02-08 22:19:57,883 INFO namenode.FSImageFormatProtobuf: Saving image file /home/hadoop/hadoopdata/hdfs/namenode/current/fsimage.ckpt_0000000000000000000
0 using no compression
2025-02-08 22:19:57,979 INFO namenode.FSImageFormatProtobuf: Image file /home/hadoop/hadoopdata/hdfs/namenode/current/fsimage.ckpt_0000000000000000000 of si
ze 401 bytes saved in 0 seconds .
2025-02-08 22:19:57,986 INFO namenode.NNStorageRetentionManager: Going to retain 1 images with txid >= 0
2025-02-08 22:19:58,016 INFO namenode.FSNamesystem: Stopping services started for active state
2025-02-08 22:19:58,017 INFO namenode.FSNamesystem: Stopping services started for standby state
2025-02-08 22:19:58,020 INFO namenode.FSImage: FSImageSaver clean checkpoint: txid=0 when meet shutdown.
2025-02-08 22:19:58,020 INFO namenode.NameNode: SHUTDOWN_MSG:
/************************************************************
SHUTDOWN_MSG: Shutting down NameNode at lab1-aryan-shivali/10.3.34.219
************************************************************/
hadoop@lab1-aryan-shivali:~/hadoop$ start-all.sh
WARNING: Attempting to start all Apache Hadoop daemons as hadoop in 10 seconds.
WARNING: This is not a recommended production deployment configuration.
WARNING: Use CTRL-C to abort.
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [lab1-aryan-shivali]
lab1-aryan-shivali: Warning: Permanently added 'lab1-aryan-shivali' (ED25519) to the list of known hosts.
Starting resourcemanager
Starting nodemanagers
hadoop@lab1-aryan-shivali:~/hadoop$ jps
146880 SecondaryNameNode
146693 DataNode
146600 NameNode
147148 NodeManager
147053 ResourceManager
147469 Jps
hadoop@lab1-aryan-shivali:~/hadoop$
```

Finding the IP Address

To find the IP address required for accessing Hadoop web interfaces, we installed net-tools and used ifconfig command.

```
hadoop@lab1-aryan-shivali:~$ ifconfig
br-e43f5c3d8a4a: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.18.0.1  netmask 255.255.0.0  broadcast 172.18.255.255
        inet6 fe80::42:bdff:fe9f:7bca  prefixlen 64  scopeid 0x20<link>
        ether 02:42:bd:9f:7b:ca  txqueuelen 0  (Ethernet)
        RX packets 81  bytes 8618 (8.6 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 228  bytes 36665 (36.6 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:e4:71:7c:11  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

ens3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 8900
        inet 10.3.34.219  netmask 255.255.255.0  broadcast 10.3.34.255
        inet6 fe80::f816:3eff:fe64:b388  prefixlen 64  scopeid 0x20<link>
        ether fa:16:3e:64:b3:88  txqueuelen 1000  (Ethernet)
        RX packets 104396  bytes 2109113174 (2.1 GB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 82548  bytes 11848244 (11.8 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Verification and HDFS Directories: -

To verify the successful installation of Hadoop, we created test directories test1 and logs in HDFS:

hdfs dfs -mkdir /test1
hdfs dfs -mkdir /logs

A screenshot confirming the creation of these directories has been attached.

```
hadoop@lab1-aryan-shivali:~$ hdfs dfs -mkdir /test1
hadoop@lab1-aryan-shivali:~$ hdfs dfs -mkdir /logs
hadoop@lab1-aryan-shivali:~$ hdfs dfs -ls /
Found 2 items
drwxr-xr-x   - hadoop supergroup          0 2025-02-08 23:05 /logs
drwxr-xr-x   - hadoop supergroup          0 2025-02-08 23:05 /test1
hadoop@lab1-aryan-shivali:~$
```

Configuring MapReduce: -

MapReduce is a programming model for processing large datasets in a distributed environment. It consists of two phases: the Map phase, which processes input data in parallel, and the Reduce phase, which aggregates results. This allows for efficient data handling across multiple nodes in a Hadoop cluster.

Running the MapReduce Example: -

We executed the WordStandardDeviation.java MapReduce program, which calculates the number of words and their statistical variations in a distributed manner. The program reads an input file, tokenizes words, and calculates the length and squared length of each word before performing aggregations in the reduce phase. This demonstrates the power of distributed processing using MapReduce.

The WordStandardDeviation program was compiled and executed successfully. We created an input file input.txt in Hadoop's directory and ran it using MapReduce. The output results have been verified and are shown in the attached screenshot.

```
hadoop@lab1-aryan-shivali:~/hadoop$ hdfs dfs -cat /output/part-r-00000
hadoop  1
hello   2
world   1
hadoop@lab1-aryan-shivali:~/hadoop$
```

Conclusion: -

This report outlines the steps taken to set up a simple Hadoop cluster on Jetstream2, along with MapReduce execution. Additionally, we have shelved our instance to ensure optimal resource management and security within Jetstream2